



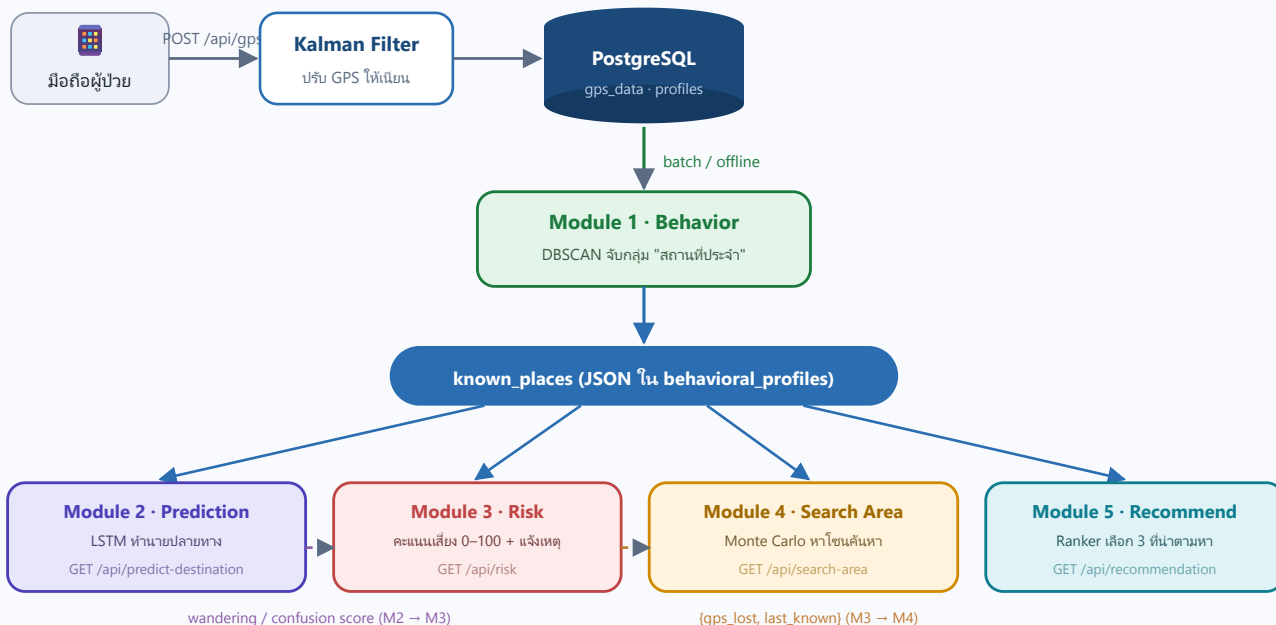
PathGuard AI

Workflow ของระบบ & AI ในแต่ละโมดูล — ตัวไหน "เรียนรู้" จริง ตัวไหนเป็นกติกา และทำไม

ระบบตรวจจับการเดินหลงของผู้ป่วยสมองเสื่อมด้วย GPS · Backend: FastAPI + PostgreSQL + Firebase · กรกฎาคม 2026

ภาพรวม: จาก GPS หนึ่งจุด → คำแนะนำถึงผู้ดูแล

ทั้ง 5 โมดูล **ไม่ได้เรียกกันตรงๆ** ตอนรันจริง แต่สื่อสารกันผ่าน 2 ช่องทาง: **(1) PostgreSQL** เป็น "บัซกลาง" — Module 1 เขียนโปรไฟล์พฤติกรรม, Module 2-5 อ่านไปใช้ และ **(2) import ฟังก์ชันบริสุทธิ์** ข้ามโมดูลเฉพาะจุด (เช่น M3 ยืม detector ของ M2)



รูปที่ 1 — สถาปัตยกรรมรวม: GPS ไหลเข้า PostgreSQL, Module 1 กลั่นเป็น known_places แล้วแจกให้ Module 2-5 · เส้นประ = การส่งข้อมูลข้ามโมดูลโดยตรง

🔑 กฎเชื่อมทั้งระบบ: `cluster_id`

Module 1 สร้าง `cluster_id` (หมายเลขสถานที่ประจำ เช่น 0=บ้าน, 1=ตลาด) แล้วทุกโมดูลใช้เลขเดียวกันนี้เป็นภาษากลาง — M2 ใช้เป็นคลาสของ LSTM, M3 ใช้เช็คความคุ้นเคย, M4 ใช้เป็นเป้าหมายค้นหา, M5 ใช้เป็นรายการจัดอันดับ

ข้อมูลที่ไหลระหว่างโมดูล	จาก → ไป	ผ่านช่องทาง
<code>known_places</code> — รายการสถานที่ประจำ (JSON)	M1 → M2, M3, M4, M5	PostgreSQL (บัซหลัก)
<code>wandering_score</code> , <code>confusion</code> (0-1)	M2 → M3	import ฟังก์ชันตรง
<code>{gps_lost, gap_seconds, last_known}</code>	M3 → M4	import ฟังก์ชันตรง
<code>RiskScore</code> , <code>Alert</code>	M3 → ฐานข้อมูล/ผู้ดูแล	PostgreSQL + Firebase

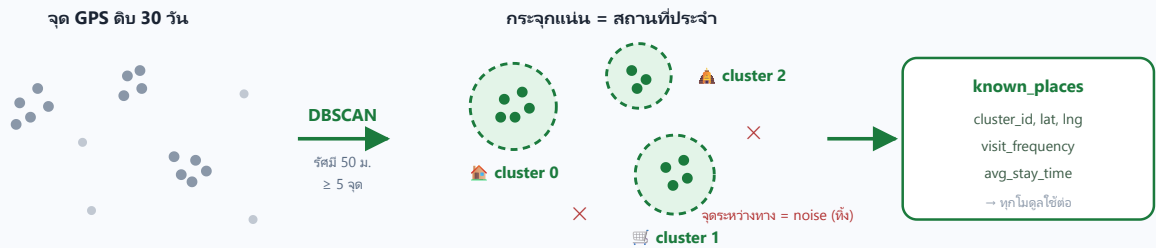
เจาะลึกทีละโมดูล: ตัวไหนเป็น AI จริง และทำไม

1

Behavior — เรียนรู้ "ความปกติ" ของผู้ป่วย

DBSCAN clustering + Kalman filter · `behavior_pipeline.analyze_behavior()`

✓ ML จริง (Unsupervised)



รูปที่ 2 — DBSCAN "มอง" จุด GPS แล้วหากระจุกเอง โดยไม่ต้องมีใครบอกว่าที่ไหนคือบ้าน

- เป็น ML แบบ **ไม่มีครูสอน (unsupervised)** — ไม่มี label ว่า "นี่คือบ้าน" มันค้นพบเองจากความหนาแน่นของจุด
- ก่อน clustering มี **Kalman filter** ปรับ GPS เด็งๆ ให้เรียบก่อน (อันนี้เป็นคณิตศาสตร์สถิติ ไม่ใช่ AI)

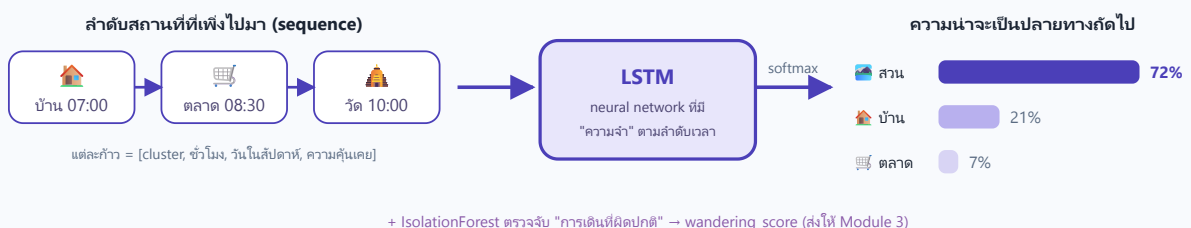
ทำไม DBSCAN ไม่ใช่ K-Means? เพราะเราไม่รู้ล่วงหน้าว่าผู้ป่วยมีสถานที่ประจำกี่แห่ง (K-Means ต้องระบุจำนวนกลุ่มก่อน) และ DBSCAN หักจุด "ระหว่างเดินทาง" เป็น noise ได้เองโดยธรรมชาติ

2

Prediction — ตอนนี้จะกำลังจะไปไหน

LSTM neural network + IsolationForest · `DestinationPredictor.predict()`

✓ ✓ Deep Learning



รูปที่ 3 — LSTM อ่าน "ลำดับ" สถานที่+เวลา แล้วทายว่าก้าวต่อไปคืออะไร · โมเดลแยกผู้ป่วย

- **Supervised learning** — เทรนจากประวัติจริงของผู้ป่วยแต่ละคน (`lstm_patient_{id}.h5`) เพราะพฤติกรรมแต่ละคนไม่เหมือนกัน
- มี AI ตัวรอง: **IsolationForest** ตรวจจับความผิดปกติ ("จุดที่แปลกจะถูกแยกจากกลุ่มได้ง่าย") → `wandering_score` 0-1
- ส่วน `StopConfusionClassifier` ชื่อเหมือน ML แต่จริงๆ เป็น rule-based (หยุดนานผิดปกติ + วนไปวนมา = สับสน)

ทำไมต้อง LSTM? เพราะ "ที่จะไปต่อ" ขึ้นกับ *ลำดับ* ของที่ที่ไปมาก่อนหน้า — โมเดลธรรมดามองเห็นแค่จุดเดียว แต่ LSTM มีเซลล์ความจำที่เก็บบริบทของทั้งเส้นทางไว้

3

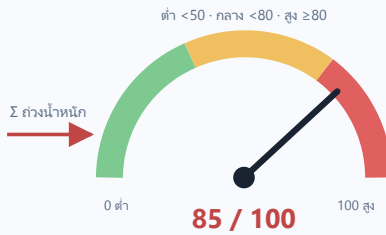
Risk — คะแนนเสี่ยง 0-100 + ตัดสินใจแจ้งเหตุ

Weighted sum (น้ำหนักเลือกโดยมนุษย์) · `calculate_risk()` + `decide_emergency()`

✗ ไม่ใช่ ML — กติกาโปรแกรม (งใจ)

5 ปัจจัย (normalize เป็น 0-1)

เบี่ยงจากเส้นทางปกติ	×0.30
เดินหลง (AI จาก M2) 🚶	×0.25
สับสน (AI จาก M2) 🚶	×0.20
อยู่ในโซนอันตราย	×0.15
พื้นที่ไม่คุ้นเคย	×0.10



Emergency Engine

if โซนอันตราย OR
risk_score > 80

🚨 แจ้งผู้ดูแลทันที

รูปที่ 4 — สูตรถ่วงน้ำหนักที่เลือกไว้ในโค้ด · 🚶 = ปัจจัยที่ "วัตถุติด" มาจาก AI ของ Module 2

ทำไมจึงใจไม่ใช้ ML ตรงนี้? เพราะจุดนี้ตัดสินใจแจ้งเหตุฉุกเฉิน — ระบบ safety-critical ต้องอธิบายได้ว่าทำไมคะแนน 85 ถึงเตือน ถ้าเป็น black-box แล้วพลาด จะตอบครอบครัวผู้ป่วยไม่ได้ อีกทั้งไม่มีข้อมูล "เหตุการณ์หลงจริงที่มี label" มากพอจะเทรนอย่างปลอดภัย · ความฉลาดของ M3 คือการเป็น "ผู้พิพากษาที่ฟังพยาน AI แล้วตัดสินใจด้วยกติกาโปรแกรม"

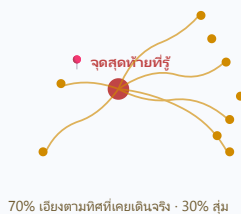
4

Search Area — GPS หายแล้ว จะไปตามหาที่ไหน

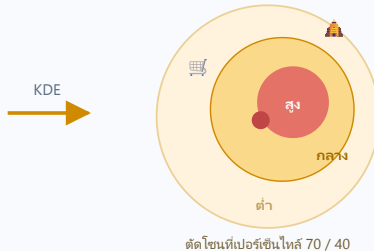
Monte Carlo simulation ×10,000 + Gaussian KDE · `PathSimulator.simulate_paths()`

— AI เชิงจำลอง (ไม่เทรน)

จำลองเดิน 10,000 ครั้ง



แผนที่ความน่าจะเป็น



หาค้นหาได้

- โซนสูง/กลาง/ต่ำ
- สถานที่เป้าหมาย (คุ้นเคย)
- เส้นทางที่น่าจะใช้

รูปที่ 5 — รัศมี = ความเร็วเดิน (1.4 m/s) × เวลาที่หายไป แล้วจำลองหมินเส้นทาง กลายเป็นโซนความน่าจะเป็น

ทำไมจำลองแทนที่จะเทรนโมเดล? เพราะอนาคตหลัง GPS หาย *ไม่มีทางรู้แน่* — แทนที่จะเดาครั้งเดียว ก็จำลอง 10,000 ครั้งแล้วดูว่าจุดจบไปกองอยู่ตรงไหน (วิธีคิดเดียวกับพยากรณ์เส้นทางพายุ) · ข้อมูลจริงยังมีบทบาท: 70% ของทิศทางจำลองเอนเอียงตามทิศที่ผู้ป่วยเคยเดินจริง

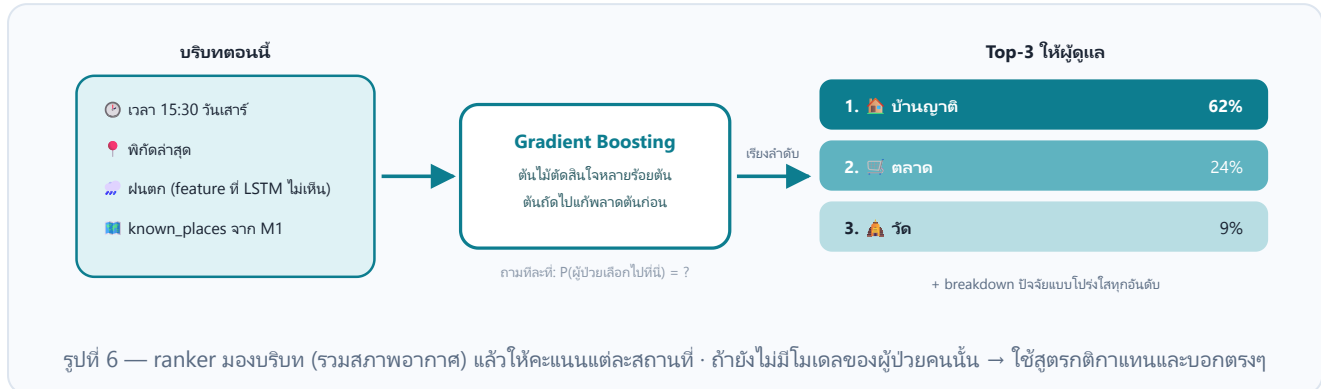
5

Recommend — บอกผู้ดูแล 3 ที่ที่น่าตามหา ก่อน

HistGradientBoosting learned ranker + rule fallback ·

`generate_recommendations()`

✓ ML จริง (Supervised Ranking)



- โจทย์เป็น **ranking** — แปลงเป็น classification: "(บริบทตอนนี้, สถานที่ X) → ผู้ป่วยเลือกไปจริงไหม?" แล้วเอา $P(\text{chosen})$ มาเรียง
- จุดต่างจาก M2: เห็น **สภาพอากาศ** (ฝนตก → ไม่น่าไปสวนสาธารณะ) ซึ่ง LSTM ไม่เห็น
- **ระบบข้อสัถย:** มี fallback เป็นสูตรกติกา ($\text{freq } 0.45 + \text{proximity } 0.35 + \text{familiarity } 0.20$) พร้อมแจ้งชัดว่าใช้ fallback อยู่ และมี evaluation harness พิสูจน์ว่า ranker ชนะ baseline +14.8% ก่อนถึงกล้าใช้

ทำไม Gradient Boosting ไม่ใช่ Neural Net? ข้อมูลเป็นแบบตาราง (ความถี่, ระยะทาง, ชั่วโมง, อากาศ) และมีจำนวนไม่มาก — สถานการณ์แบบนี้ต้นไม้ boosting มักชนะ neural network ทั้งความแม่นยำและความเสถียร

สรุป: แผนที่ AI ของทั้งระบบ

Module	เทคนิคหลัก	"เรียนรู้" จริงไหม	เหตุผลที่เลือกแบบนี้
1 Behavior	DBSCAN + Kalman	✓ unsupervised	ไม่รู้จำนวนสถานที่ล่วงหน้า, หัก noise ได้เอง
2 Prediction	LSTM + IsolationForest	✓ ✓ deep learning	ปลายทางขึ้นกับลำดับพฤติกรรม
3 Risk	weighted sum + if-else	✗ กติกาคนกำหนด	safety-critical ต้องอธิบายได้ · ไม่มี label ให้เทรน
4 Search Area	Monte Carlo + KDE	— จำลอง ไม่เทรน	อนาคตไม่แน่นอน → จำลอง همینครั้งแทนเดาครั้งเดียว
5 Recommend	Gradient Boosting ranker	✓ supervised + fallback	ข้อมูลตาราง+น้อย → boosting ชนะ neural net

ธีมของสถาปัตยกรรม: ให้ AI เรียนรู้ในจุดที่พฤติกรรมส่วนบุคคลสำคัญ (M1, M2, M5) ใช้กติกาโปร่งใสในจุดที่ชีวิตคนแขวนอยู่ (M3) และใช้ความน่าจะเป็นในจุดที่ *ไม่มีข้อมูลจะรู้ได้* (M4)

⚠️ ช่องว่างที่ยังต้องเชื่อม (สถานะปัจจุบันของโค้ด)

- `POST /api/gps` ยังเขียนลง **list** ในหน่วยความจำ ไม่ใช่ PostgreSQL — ท่อ `gps_processor` + Kalman ที่ต่อ DB จริงเขียนเสร็จแล้วแต่ยังไม่มี router เรียก
- ยังไม่มี endpoint ส่งเทรน Module 1 (`analyze_behavior`) — โปรไฟล์ต้อง seed เอาผ่าน `mock/seed_module5.py`
- `time_match` ใน M5 และ `wandering_score` ที่จะเข้า M4 ยัง stub อยู่ รอ Module 1 เขียน `routine_patterns`

